

Tech Challenge — Fase 4 — Entrega no portal do aluno (PDF)

Grupo: Oficina Turbo (106)
Aluno: Felipe Ricarte Magalhães

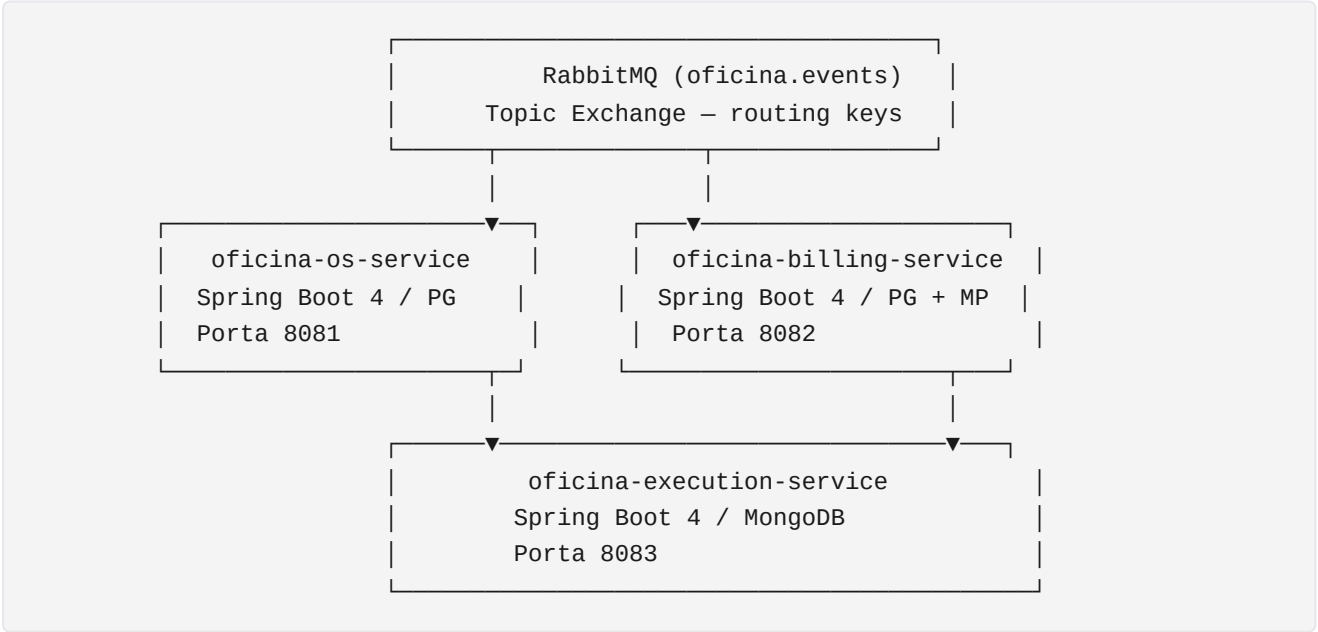
1. Três repositórios de microsserviços (Fase 4)

#	Repositório	Responsabilidade	Banco
1	https://github.com/ricartefelipe/oficina-os-service	Ordens de Serviço — abertura, status, histórico	PostgreSQL
2	https://github.com/ricartefelipe/oficina-billing-service	Orçamentos e pagamentos via Mercado Pago	PostgreSQL
3	https://github.com/ricartefelipe/oficina-execution-service	Fila de execução técnica	MongoDB

Branch principal: `main` (CI/CD ativo).
Acesso ao avaliador SOAT: usuário `soat-architecture` convidado com leitura nos três repositórios.

2. Arquitetura — Microsserviços e Saga Pattern

2.1 Visão geral



2.2 Saga Pattern — Coreografia

Cada evento flui de forma assíncrona pelo broker. Não há orquestrador central.

Evento (routing key)	Publicado por	Consumido por	Resultado
os.aberta	OS Service	Billing Service	Gera orçamento
orcamento.gerado	Billing Service	—	(informativo)
orcamento.aprovado	Billing Service	OS Service	OS → PAGAMENTO_PENDENTE
orcamento.recusado	Billing Service	OS Service	OS → CANCELADA (compensação)
pagamento.confirmando	Billing Service	OS Service + Execution Service	OS → EM_EXECUCAO; Execution → NA_FILA
pagamento.falhou	Billing Service	OS Service + Execution Service	OS → CANCELADA (compensação)
execucao.finalizada	Execution Service	OS Service + Billing Service	OS → FINALIZADA; Billing encerra cobrança

2.3 Justificativa: Coreografia vs. Orquestração

Adotamos **coreografia** porque:

- Cada serviço conhece apenas seu próprio domínio — baixo acoplamento
- Não há ponto único de falha (sem orquestrador central)
- Escalabilidade independente de cada consumidor
- Compensações são tratadas pelos próprios serviços (ex: OS cancela ao receber `orcamento.recusado`)

3. Stack técnica

Componente	Tecnologia
Linguagem	Java 21
Framework	Spring Boot 4.0.6
Mensageria	RabbitMQ + Spring AMQP
Banco relacional	PostgreSQL (OS Service, Billing Service)
Banco NoSQL	MongoDB (Execution Service)
Migrations	Liquibase
Pagamentos	Mercado Pago SDK 3.2.1
Segurança	OAuth2 / JWT (Keycloak)
Testes	JUnit 5 + Cucumber (BDD) + Mockito

Componente	Tecnologia
Cobertura	JaCoCo $\geq 80\%$
Qualidade	SonarCloud
CI/CD	GitHub Actions
Containers	Docker + docker-compose (infra local)
Kubernetes	Manifests em /k8s (Deployment, Service, ConfigMap, HPA)
Observabilidade	Micrometer + Prometheus + Grafana

4. Cobertura de testes (JaCoCo)

Serviço	Cobertura	Status
oficina-os-service	92.9%	✓ $\geq 80\%$ — SonarCloud Quality Gate: OK
oficina-billing-service	84.5%	✓ $\geq 80\%$ — SonarCloud Quality Gate: OK
oficina-execution-service	93.0%	✓ $\geq 80\%$ — SonarCloud Quality Gate: OK

Tipos de teste implementados:

- **Unitários** — domínio, casos de uso, adaptadores, listeners, controllers
- **BDD / Cucumber** — cenários em português para os fluxos principais
- **Contexto** — `@SpringBootTest` validando boot completo com H2/mock-MongoDB

5. CI/CD — GitHub Actions

Cada repositório possui dois workflows:

Workflow	Gatilho	Etapas
<code>ci.yml</code>	Push em qualquer branch / PR	Checkout → Java 21 → <code>mvnw verify</code> (testes + JaCoCo) → Upload relatório → SonarCloud (só <code>main</code>) → Build e push Docker (GHCR)
<code>deploy.yml</code>	<code>workflow_dispatch</code> (manual)	Kubectl apply → Atualiza imagem no cluster K8s

Imagens publicadas em: `ghcr.io/ricartefelipe/<serviço>:latest`

6. Como executar localmente

6.1 Pré-requisitos

- Docker + Docker Compose
- Java 21 + Maven Wrapper (`./mvnw`)

6.2 Subir infraestrutura compartilhada

```
# No repositório oficina-os-service (tem o docker-compose.infra.yml)
docker compose -f docker-compose.infra.yml up -d
```

Sobe: RabbitMQ (15672), PostgreSQL-OS (5432), PostgreSQL-Billing (5433), MongoDB (27017), Keycloak (8080), Prometheus (9090), Grafana (3000).

6.3 Rodar cada serviço

```
# OS Service
cd oficina-os-service && ./mvnw spring-boot:run

# Billing Service
cd oficina-billing-service && ./mvnw spring-boot:run

# Execution Service
cd oficina-execution-service && ./mvnw spring-boot:run
```

6.4 Swagger UI

Serviço	URL
OS Service	http://localhost:8081/swagger-ui.html
Billing Service	http://localhost:8082/swagger-ui.html
Execution Service	http://localhost:8083/swagger-ui.html

6.5 Rodar todos os testes

```
./mvnw -Pci clean verify # em cada repositório
```

7. Kubernetes

Cada repositório contém `/k8s` com:

Arquivo	Conteúdo
<code>namespace.yaml</code>	Namespace <code>oficina</code>

Arquivo	Conteúdo
configmap.yaml	Variáveis de ambiente não sensíveis
secret.example.yaml	Exemplo de Secret (DB password, tokens)
deployment.yaml	Deployment + Service + HPA (min 1, max 3 réplicas)

Deploy manual:

```
kubectl apply -f k8s/
kubectl set image deployment/<serviço> app=ghcr.io/ricartefelipe/<serviço>:<sha>
```

8. Observabilidade

Reutilizada da Fase 3 — Prometheus + Grafana já sobem via `docker-compose.infra.yaml`.

Ferramenta	Acesso
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 (admin/admin)
RabbitMQ Management	http://localhost:15672 (guest/guest)

Métricas expostas via `/actuator/prometheus` em cada serviço.

9. Vídeo demonstrativo (≤ 15 minutos)

Link: (inserir URL YouTube/Vimeo antes do envio no portal)

Roteiro sugerido:

1. `docker compose -f docker-compose.infra.yaml up -d` — infra local no ar
2. Subir os 3 serviços
3. `POST /admin/ordens-servico` — abrir OS → evento `os.aberta` no RabbitMQ Management
4. Billing gera orçamento → aprovar via `POST /admin/orcamentos/{osId}/aprovar`
5. Pagamento confirmado → Execution Service adiciona OS à fila
6. Técnico inicia diagnóstico e finaliza
7. OS chega ao status `FINALIZADA`
8. GitHub Actions rodando (CI verde nos 3 repos)
9. Prometheus/Grafana com métricas

10. Checklist de requisitos (enunciado Fase 4)

Requisito	Status
≥ 3 microsserviços independentes	✓ OS, Billing, Execution
Banco relacional em ≥ 1 serviço	✓ PostgreSQL em OS e Billing
Banco NoSQL em ≥ 1 serviço	✓ MongoDB em Execution
Comunicação via mensageria assíncrona	✓ RabbitMQ — Topic Exchange
Comunicação síncrona (RESTful)	✓ Endpoints REST em cada serviço
Saga Pattern implementado	✓ Coreografia com compensações
Testes unitários	✓ JUnit 5 + Mockito
Testes BDD	✓ Cucumber em todos os serviços
Cobertura ≥ 80% (JaCoCo)	✓ 92.9% / 84.5% / 93.0%
SonarCloud Quality Gate	✓ OK nos 3 serviços — 0 bugs, 0 vulnerabilidades abertas
CI/CD automatizado	✓ GitHub Actions (<code>ci.yml</code> + <code>deploy.yml</code>) em todos os repos
Deploy Kubernetes	✓ /k8s completo em todos os repos (namespace, configmap, secret, deployment)
Observabilidade (Fase 3 reutilizada)	✓ Prometheus + Grafana via <code>docker-compose.infra.yml</code>
Vídeo demonstrativo	🕒 Pendente — a ser inserido antes do envio no portal